might contain the following data:

```
depot> sqlite3 -line storage/development.sqlite3 "select * from orders limit
1"
        id = 1
      name = Dave Thomas
   address = 123 Main St
     email = customer@example.com
  pay_type = 0
created_at = 2022-02-14 14:39:12.375458
updated_at = 2022-02-14 14:39:12.375458
```

If we fetched this row into an Active Record object, that object would have seven attributes. The `id` attribute would be `1` (an `Integer`), the `name` attribute would be the string `"Dave Thomas"`, and so on.

We access these attributes using accessor methods. Rails automatically constructs both attribute readers and attribute writers when it reflects on the schema:

```
o = Order.find(1)
puts o.name                  #=> "Dave Thomas"
o.name = "Fred Smith"        # set the name
```

Setting the value of an attribute doesn't change anything in the database—we must save the object for this change to become permanent.

The value returned by the attribute readers is cast by Active Record to an appropriate Ruby type if possible (so, for example, if the database column is a timestamp, a `Time` object will be returned). If we want to get the raw value of an attribute, we append `_before_type_cast` to its name, as shown in the following code:

```
Order.first.pay_type                  #=> "Check", a string
Order.first.pay_type_before_type_cast  #=> 0, an integer
```

Inside the code of the model, we can use the `read_attribute` and `write_attribute` private methods. These take the attribute name as a string parameter.